

CW2 - Neural Networks

Jade Carpentier, Thomas Cherqui, Hannah O’Keeffe

November 2025

1 Description of the model

1.1 Data Preprocessing

The California Housing dataset contains 16,512 samples with 9 input features (8 numerical and 1 categorical) and the target label *median_house_value*. We begin by examining the distribution of our feature of interest using the histogram shown in Figure 1. The distribution is right-skewed, with a long tail extending toward higher values. We see a large spike at \$500,000 which could correspond to higher value properties being encoded together at this upper threshold.

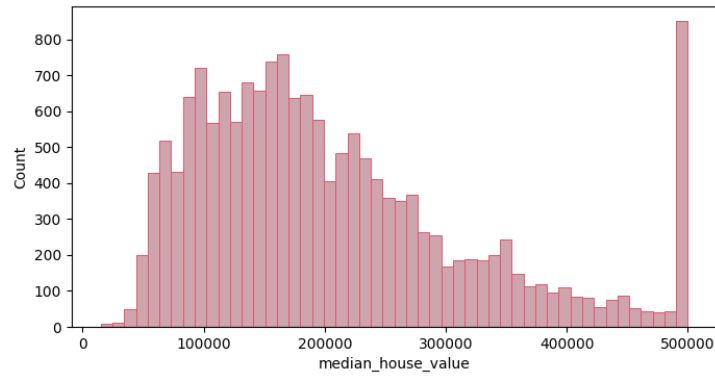


Figure 1: Distribution of the median house values

For the numerical features, only *total_bedrooms* contains missing values (168 entries). The choice between imputing with the mean or the median depends on the shape of the distribution. The mean is appropriate for roughly symmetric distributions, while the median is more robust when strong skewness or outliers are present. According to Figure 2, it is more appropriate to choose the median for this feature.

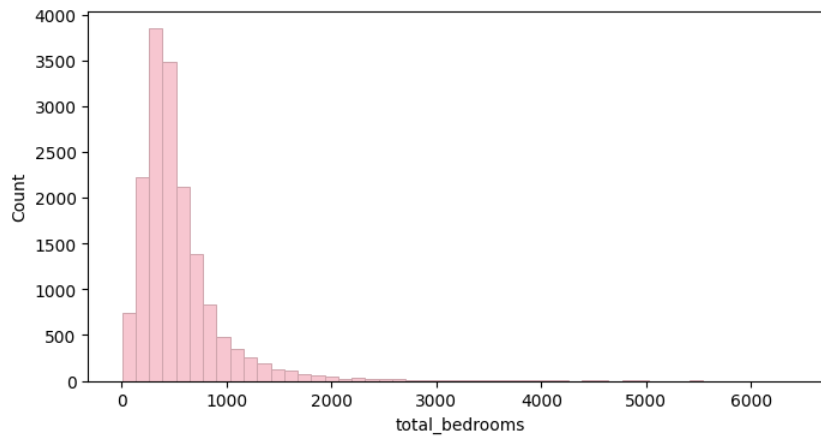


Figure 2: Distribution *total_bedrooms* feature

For the categorical feature *ocean_proximity*, we use one-hot-encoding. During training, the label binarizer is fitted on the resulting set of categories. At validation or test time, any category not observed during training is replaced with “None” to avoid unseen-category errors. After this preprocessing, *ocean_proximity* is one-hot encoded using the categories learned from the training set.

Finally, we scale both the input features and the output variable to ensure that all dimensions are expressed on comparable scales, which facilitates model training and improves numerical stability. The scaled labels allow the model to train efficiently while preserving the ability to recover the original values via inverse transformation. We use a `StandardScaler` for normalisation which subtracts the mean and divides by the standard deviation for each feature. Before computing evaluation metrics, we apply the inverse transformation to the model’s predictions so that both predictions and true labels are expressed in the original dollar scale.

1.2 Model Architecture

We want to determine the median house price for a housing block based on input features describing the housing block, making this a regression problem. To address this task, we implemented a Neural Network that can capture non-linear links between the input features and the output price.

Given we do not have a vast set of features, we chose to use a network with two hidden layers and ReLU activation function. This adds non-linearities while keeping the model simple enough to avoid the risk of overfitting. For the output layer, we use a linear activation function, as is standard for a regression task such as this, which returns an unrestricted numerical value. Alongside the model architecture and activation functions, the model configuration depends on several hyperparameters, including the following:

- **Learning rate:** the step size used at each gradient descent update.
- **Batch size:** the number of training samples processed simultaneously in one forward and backward pass.

2 Evaluation Setup

2.1 Dataset split

We split the dataset as following : 80% for training, 10% for validation and 10% for testing. The validation set is used for hyperparameter tuning, while the test set is used once, at the end, to evaluate the final model.

2.2 Evaluation metrics

To evaluate our model, we use several metrics. Our primary metric is Mean Squared Error (MSE):

$$\text{MSE} = \frac{1}{N} \sum_{i=1}^N (y_i - \hat{y}_i)^2.$$

MSE penalises large errors more heavily due to the squared term, making it particularly sensitive to outliers. It is used both as the loss function during training and as the main criterion for selecting hyperparameters based on validation performance.

While optimisation is performed using MSE, we interpret performance using the Root Mean Squared Error (RMSE) which is a monotonic transformation of MSE, meaning it preserves the ordering of the models. The configuration with the lowest MSE will always also have the lowest RMSE.

In addition, we identified a number of other evaluation metrics to provide a deeper understanding of the performance of the model, as detailed below:

Coefficient of Determination (R^2) :

$$R^2 = 1 - \frac{\sum_{i=1}^N (y_i - \hat{y}_i)^2}{\sum_{i=1}^N (y_i - \bar{y})^2},$$

An R^2 of 1 indicates perfect predictions, $R^2 = 0$ means the model performs no better than predicting the mean.

Mean Absolute Error (MAE) :

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |y_i - \hat{y}_i|.$$

MAE provides an interpretable measure of average error in absolute dollar terms, making it useful for understanding typical prediction deviations, without penalising large outliers.

Mean Absolute Percentage Error (MAPE) :

$$\text{MAPE} = \frac{1}{N} \sum_{i=1}^N \frac{|y_i - \hat{y}_i|}{y_i} * 100$$

MAPE expresses prediction error as a percentage of the true value allowing us to assess how performance varies across different price ranges (as in Section 4.2).

3 Hyper-parameter search

3.1 Methodology

The performance of the neural network depends on several key hyperparameters, including the learning rate η which controls the step size in gradient descent, and the batch size which sets the number of samples used to compute each gradient update. We perform a grid search over these parameters to identify stable training configurations. Each combination is trained on the training data set and evaluated on the validation set. Tested parameters are summarized in Table 1.

Hyperparameter	Value
Learning rate (η)	2×10^{-4} , 2×10^{-3} , 2×10^{-2}
Batch size	64, 128

Table 1: Hyperparameters tested

3.2 Results

Figure 3 shows the training loss (reported using de-scaled mean squared error) over 200 epochs for each hyperparameter configuration.

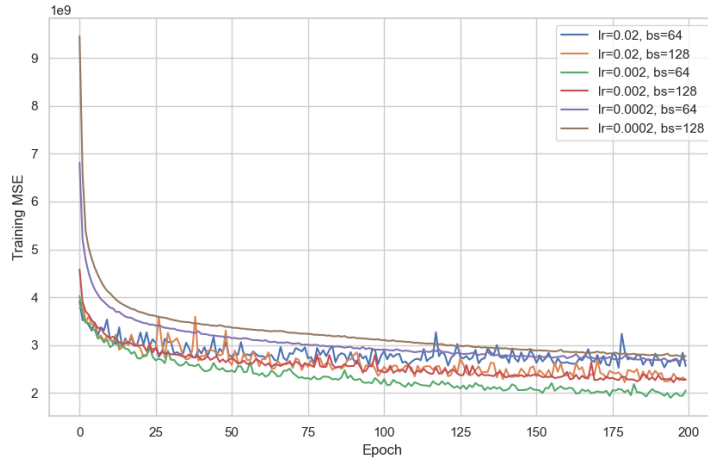


Figure 3: Training Loss convergence for different hyper-parameters settings

Configuration	Validation MSE	Validation RMSE	Validation R^2
lr = 0.02, bs = 64	2 696 255 257.80	51 925.48	0.8027
lr = 0.02, bs = 128	2 688 367 035.79	51 849.47	0.8033
lr = 0.002, bs = 64	2 747 625 455.34	52 417.80	0.7989
lr = 0.002, bs = 128	2 682 546 394.87	51 793.30	0.8037
lr = 0.0002, bs = 64	2 796 910 996.50	52 885.83	0.7953
lr = 0.0002, bs = 128	2 866 648 395.98	53 541.09	0.7902

Table 2: Validation metrics for each configuration of hyperparameters (reported using de-scaled metrics)

We can see from Figure 3 that all models rapidly reduce their loss within the first 25 epochs, with most configurations plateauing around 100 epochs. Although the different configurations achieve broadly similar validation RMSE values, the best-performing model is the one using a learning rate of $\eta = 0.002$ and a batch size of $bs = 128$, as highlighted in Table 2.

4 Evaluation of the best model

4.1 Overall Results

The results from our hyperparameter search indicate the best-performing model uses a learning rate of $\eta = 0.002$ and a batch size of $bs = 128$. We now evaluate this selected model on the held-out test data and obtain the following results:

Test MSE (\$)	Test RMSE (\$)	Test R^2
2 808 674 912.99	52 996.93	0.7931

Table 3: Final Results on Best-Performing Model

We use the RMSE to better interpret the performance of the model. This tells us that on average our predictions are around \$53,000 away from the true value of the house, although RMSE does penalise larger errors due to the squared term. The R^2 value of 0.793 also indicates that this model can explain around 79% of the variance in the median house price data available in the test set. We believe this to be reasonably good predictive power.

4.2 Distribution Analysis

We evaluated model performance across subsets of the test data to understand how our errors vary across the distribution of house prices. Given the skewed distribution displayed in Figure 1, we expect larger errors for the right-hand side of the distribution, where the model sees fewer training examples.

Figure 4 shows the number of samples per bin (left) and RMSE per bin (right). As expected, the RMSE is much higher for properties valued above \$300,000, reflecting the shortage of these high-value examples and the spike at the \$500,000 mark. In contrast, performance is better for examples in the dataset that lie between \$50,000 and \$300,000 where we see a concentration of samples.

The difference between MAE and MAPE in Table 4 tells us that while the absolute error in dollars terms increases for the higher-valued properties, the relative error is not much larger in this part of the distribution. Conversely, the lower-priced homes have smaller absolute errors but larger percentage errors.

Overall, we decided to use RMSE as our primary evaluation metric given its interpretability and consistency with the loss function used during training; however, this analysis displays how different evaluation metrics can highlight different aspects of model performance and how errors can vary across the price distribution.

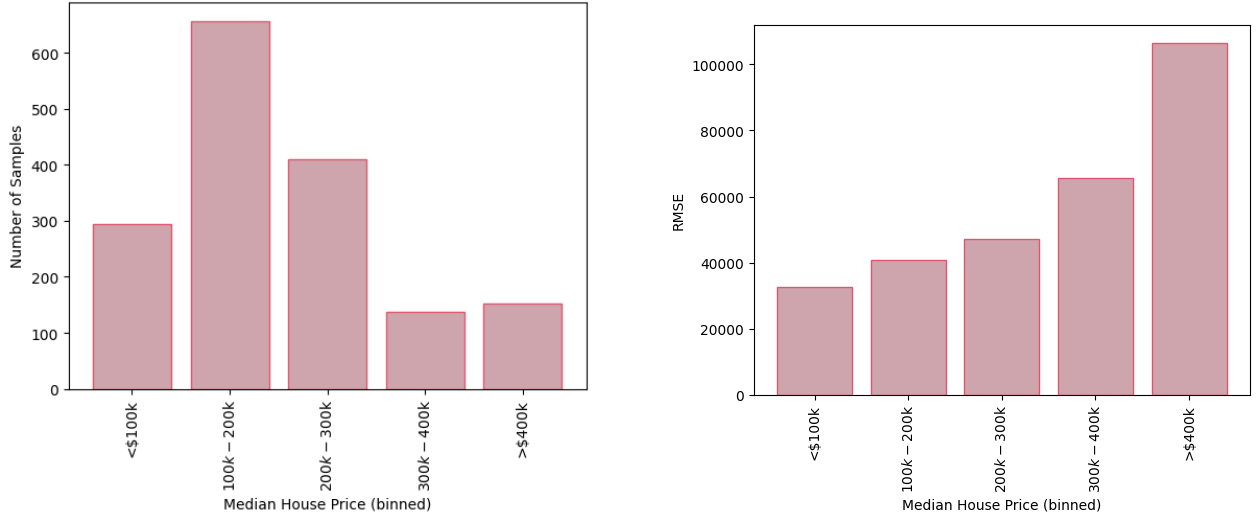


Figure 4: Figures to display distribution of errors across binned house price ranges

Price Range (\$)	Samples	RMSE (\$)	MAE (\$)	MAPE (%)
< \$100k	295	32487.78	22192.72	30.40
\$100k – \$200k	657	40667.41	27474.87	19.11
\$200k – \$300k	410	47168.66	35813.31	14.74
\$300k – \$400k	137	65687.86	51733.23	14.67
≥ \$400k	153	106601.00	79397.03	16.78

Table 4: Error metrics by price range